

Reviewer Pack — Möbius Mass of Gate-Support Meet-Semilattice Bounds AC^0 PARITY...

Entry 15f8d5df3519 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 00:24:41 UTC

Möbius Mass of Gate-Support Meet-Semilattice Bounds AC^0 PARITY Size

Entry ID: 15f8d5df3519 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 00:24:41 UTC

1. Conjecture

Field A (mathematical branch): Rota incidence algebra (Möbius functions of finite meet-semilattices) — a corner of order theory rarely formalized for circuit complexity, with < 5 papers crossing it with AC^0 or communication-matrix lower bounds **Field B** (complexity object): AC^0 circuit size for PARITY_n, viewed via the Sherstov pattern-matrix lift to deterministic two-party communication complexity (so the S_n action on inputs lifts to a row/column symmetry of the communication matrix)

Statement:

For an AC^0 circuit C on n inputs, let P_C be the meet-semilattice generated by the gate variable-supports $\{S(g) : g \in \text{gates}(C)\}$ together with $\emptyset$ and $[n]$, closed under set intersection and ordered by $\subseteq$, and let μ_{P_C} denote its Möbius function. Define the Möbius-mass invariant $\psi(C) := \log_2(1 + \sum_{S \in P_C} |\mu_{P_C}(\emptyset, S)|)$; this is naturally invariant under the S_n action permuting input variables, hence descends to the Sherstov pattern-matrix symmetry group. We conjecture that for every AC^0 circuit C of depth d that computes PARITY_n exactly, $\psi(C) \geq c \cdot n^{\frac{1}{d-1}}$ for an absolute constant $c > 0$ independent of d, n ; a single counterexample (one PARITY-correct circuit with $\psi(C)$ below this threshold) refutes it.

Rationale (proposer's reasoning):

Random restrictions are blind to the global meet-structure of gate supports, but the Möbius function on the support lattice encodes all higher-order intersection inclusion-exclusion in one rational number — exactly what shellability/Cohen-Macaulay arguments leverage. Because PARITY genuinely depends on all n inputs, any AC^0 circuit computing it must populate rank-strata of its support meet-semilattice in a way that prevents the alternating sum from collapsing, forcing $|\mu|$ -mass to accumulate as the depth tightens the fan-in budget per layer.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 092485079eeca2be

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{4, 8, 12, 16, 20, 24, 32, 40\}$, $d \in \{2, 3, 4\}$, seeds 0..29 plus canonical (≈ 31 PARITY-verified circuits per (n, d)): for each d , OLS-regress $\psi(C)$ on $n^{\{1/(d-1)\}}$. SUPPORT iff every d yields slope ≥ 0.1 AND no surviving circuit has $\psi(C) < 0.05 \cdot n^{\{1/(d-1)\}}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Möbius function incidence algebra AC0 circuit lower bound PARITY - meet-semilattice gate support circuit complexity Sherstov pattern matrix - Rota Möbius function bounded-depth circuit symmetric communication complexity PARITY

Top relevant hits considered: - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/1410.8702v3] The Möbius function of the small Ree groups - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/0805.2135v1] Communication Lower Bounds Using Dual Polynomials - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2508.04609v2] Analog Solver Circuit for Linear Symmetric Positive-Definite Systems at a Complexity Independent of Matrix Size - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
```

```

        return b, 0, 1
    else:
        g, x, y = extended_gcd(b % a, a)
        return g, y - (b // a) * x, x

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Inverse doesn't exist")
    else:
        return x % m

def matrix_mul(A, B):
    n = len(A)
    result = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_inv(A):
    n = len(A)
    det = 0
    if n == 1:
        det = A[0][0]
    elif n == 2:
        det = A[0][0] * A[1][1] - A[0][1] * A[1][0]
    else:
        for c in range(n):
            sub_matrix = [row[:c] + row[c+1:] for row in A[1:]]
            sign = (-1) ** (c % 2)
            sub_det = matrix_det(sub_matrix)
            det += sign * A[0][c] * sub_det
    if det == 0:
        raise ValueError("Matrix is not invertible")
    inv_det = mod_inverse(det, n)
    adjugate = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            sub_matrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            sign = (-1) ** ((i+j) % 2)
            adjugate[i][j] = sign * matrix_det(sub_matrix)
    inv_A = [[(adjugate[i][j] * inv_det) % n for j in range(n)] for i in range(n)]
    return inv_A

def matrix_det(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    elif n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    else:
        det = 0
        for c in range(n):
            sub_matrix = [row[:c] + row[c+1:] for row in A[1:]]

```

```

        sign = (-1) ** (c % 2)
        det += sign * A[0][c] * matrix_det(sub_matrix)
    return det

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0 for _ in range(n)]
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 8, 12, 16, 20, 24, 32, 40]
    d_values = [2, 3, 4]
    c = 0.05
    results = []

    for n in n_values:
        for d in d_values:
            if n <= 20:
                inputs = list(range(n))
                outputs = [random.choice([0, 1]) for _ in range(2**n)]
            else:
                inputs = list(range(n))
                outputs = [random.randint(0, 1) for _ in range(512)]

            # Construct canonical PARITY circuit
            if n == 4 and d == 2:
                circuit = [[i] for i in range(n)]
            elif n == 8 and d == 3:
                circuit = [[i] for i in range(n)]
            elif n == 12 and d == 4:
                circuit = [[i] for i in range(n)]
            else:
                return {"metric_name": "psi", "metric_value": None, "instances_tested": 0, "conjecture": False}

            # Compute meet-closure P_C of gate supports
            P_C = set()
            for S in circuit:
                P_C.add(frozenset(S))
            P_C.add(frozenset())
            P_C.add(frozenset(range(n)))
            while True:

```

```

        new_elements = set()
        for S1 in P_C:
            for S2 in P_C:
                if S1.issubset(S2):
                    continue
                new_elements.add(S1.intersection(S2))
        if not new_elements:
            break
        P_C.update(new_elements)

# Evaluate  $\mu_{\{P_C\}}(\emptyset, \cdot)$  by topological recursion
mu = {frozenset(): 0}
for S in sorted(P_C, key=len):
    mu[S] = sum(mu[frozenset(T)] for T in P_C if T.issubset(S) and len(T) == len(S) - 1)

# Compute  $\psi(C)$ 
psi = math.log2(1 + sum(abs(mu[frozenset(S)]) for S in P_C))
results.append(psi)

# Perform OLS regression
n_values = [n for _ in range(len(n_values)) for _ in range(len(d_values))]
d_values = [d for _ in range(len(n_values)) for _ in range(len(d_values))]
psi_values = [psi for psi in results]

X = [[n**(1/(d-1)), 1] for n, d in zip(n_values, d_values)]
Y = psi_values

A = matrix_mul(matrix_transpose(X), X)
b = matrix_mul(matrix_transpose(X), Y)
inv_A = matrix_inv(A)
beta = matrix_mul(inv_A, b)

slope = beta[0]
intercept = beta[1]

return {
    "metric_name": "psi",
    "metric_value": slope,
    "instances_tested": len(results),
    "conjecture_holds": slope >= 0.1,
    "counterexample": "" if slope >= 0.1 else f"Counterexample found with slope {slope}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(sys.argv[1])] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        if "conjecture_holds" in result and result["conjecture_holds"]:
            results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean)**2 for x in results) / len(results))

```

```

support_fraction = len([r for r in results if r >= 0.1]) / len(results)

if all(r >= 0.1 for r in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r < 0.1 for r in results):
    first_failing_seed = seeds[results.index(min([r for r in results if r < 0.1]))]
    print(f"RESULT: FALSIFIED counterexample=\"slope_below_threshold\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out (returncode=124) after 240s without producing any RESULT line or data, so neither the support nor falsification criteria can be evaluated. | next: Reduce the parameter grid (e.g., cap n at 20 and d at 3) or precompute the meet-semilattice incrementally to fit within the time budget, then rerun the pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	426416
2	preregistration	claude_max	opus	0	0	7000
3	novelty	claude_max	opus	0	0	3860
4	novelty	claude_max	opus	0	0	9794
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23179
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	127590
7	judge	claude_max	opus	0	0	5040

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 602878 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in [research/audit/15f8d5df3519.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/15f8d5df3519.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/15f8d5df3519.tar.gz` (if generated)